

**ESCOLA ESTADUAL “MONSENHOR HORTA”
DESENVOLVIMENTO DE SISTEMAS**

Sanderson André Elias

BIBLIOTECA MONSA
Sistema para gerenciamento de acervos, empréstimos e usuários

Conselheiro Lafaiete
2026

Sanderson André Elias

BIBLIOTECA MONSA

Sistema para gerenciamento de acervos, empréstimos e usuários

Projeto de portfólio apresentado por Sanderson André Elias em demonstração de competências em desenvolvimento de software e proposta de implantação na Escola Estadual “Monsenhor Horta”.

Orientador: Prof. Willian Chaves Ferreira

Conselheiro Lafaiete

2026

RESUMO

O presente trabalho descreve o desenvolvimento do sistema Biblioteca Monsa, uma aplicação web full-stack destinada à gestão de acervos, exemplares, empréstimos, reservas e usuários em bibliotecas escolares. A solução adota arquitetura cliente-servidor, com backend em Spring Boot 3.5.11 sobre Java 21 e frontend em React 18 com TypeScript. A persistência é realizada em PostgreSQL, com controle de versão de schema por meio do Flyway. A autenticação baseia-se em tokens JWT, e o controle de acesso é segmentado em três perfis: Administrador, Funcionário e Aluno. O sistema contempla funcionalidades como catalogação de livros com controle individual de exemplares, fluxo de solicitação e aprovação de renovações de empréstimos, fila de reservas com expiração automática, e rotinas agendadas para marcação de empréstimos em atraso. O frontend encontra-se hospedado na Vercel e o backend na Railway. O código-fonte está disponível publicamente em repositório no GitHub, para fins acadêmicos e de estudo.

Palavras-chave: Sistema bibliotecário; Spring Boot; React; API REST; Autenticação JWT; Gestão de exemplares.

SUMÁRIO

RESUMO	3
SUMÁRIO	4
LISTA DE FIGURAS	5
LISTA DE QUADROS	6
LISTA DE ABREVIATURAS E SIGLAS	7
1 INTRODUÇÃO	8
2 DESENVOLVIMENTO	10
2.1 Especificação do Projeto	10
2.1.1 Perfis de usuário	10
2.1.2 Funcionalidades principais	10
2.1.3 Fluxo de empréstimo	11
2.1.4 Rotinas agendadas	12
2.2 Arquitetura do Sistema	12
2.2.1 Organização do back-end	13
2.2.2 Autenticação e controle de acesso	13
2.2.3 Organização do front-end	14
2.3 Tecnologias Utilizadas	15
2.3.1 Back-end	15
2.3.2 Front-end	15
2.3.3 Banco de dados e infraestrutura	16
2.4 Modelo de Dados	16
2.4.1 Tabelas do domínio	17
2.4.2 Relacionamentos e integridade	18
2.4.3 Migração retroativa na introdução de exemplares	19
2.5 API e Endpoints	19
2.5.1 Autenticação	19
2.5.2 Acervo (livros e entidades relacionadas)	19
2.5.3 Exemplares	20
2.5.4 Empréstimos	20
2.5.5 Solicitações de renovação	20
2.5.6 Reservas	21
2.6 Metodologia de Desenvolvimento	21
2.7 Implantação	22
2.7.1 Ambiente de desenvolvimento	22
2.7.2 Ambiente de produção	22
3 CONSIDERAÇÕES FINAIS	24
REFERÊNCIAS	26

LISTA DE FIGURAS

Figura 1 — Arquitetura em três camadas da Biblioteca Monsa12

Figura 2 — Diagrama Entidade-Relacionamento da Biblioteca Monsa16

LISTA DE QUADROS

Quadro 1 — Matriz de permissões por recurso13

Quadro 2 — Tabelas do banco de dados17

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
API	Application Programming Interface
CDD	Classificação Decimal de Dewey
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object
HTTP	Hypertext Transfer Protocol
JDBC	Java Database Connectivity
JPA	Jakarta Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
NBR	Norma Brasileira
ORM	Object-Relational Mapping
REST	Representational State Transfer
SGBD	Sistema Gerenciador de Banco de Dados
SQL	Structured Query Language
SSL/TLS	Secure Sockets Layer / Transport Layer Security
SVG	Scalable Vector Graphics
URL	Uniform Resource Locator
XML	eXtensible Markup Language

1 INTRODUÇÃO

A gestão de bibliotecas escolares apresenta desafios operacionais que afetam diretamente a experiência dos alunos e a rotina de trabalho dos funcionários responsáveis pelo acervo. Registros manuais e planilhas eletrônicas, embora ainda comuns em instituições de ensino, apresentam limitações quanto à rastreabilidade de exemplares, ao controle de prazos de devolução e à geração de relatórios consolidados. Além disso, a dependência de processos manuais tende a aumentar a probabilidade de erros de registro e a reduzir a disponibilidade da equipe para atividades de maior valor, como a orientação aos leitores.

Diante desse contexto, o presente trabalho apresenta o desenvolvimento do sistema Biblioteca Monsa, uma aplicação web voltada à automação dos processos centrais de uma biblioteca escolar. O sistema foi concebido como projeto de portfólio do autor e como proposta de implantação na Escola Estadual “Monsenhor Horta”, onde o autor cursa o Ensino Médio Técnico em Desenvolvimento de Sistemas.

O objetivo geral do projeto consiste em desenvolver uma aplicação web, com interfaces distintas para diferentes perfis de usuário, que permita o controle do acervo, o registro de empréstimos vinculados a exemplares individuais, a administração de uma fila de reservas e o gerenciamento de solicitações de renovação.

Como objetivos específicos, foram definidos: (i) modelar o domínio de uma biblioteca escolar em um banco de dados relacional, incluindo a distinção entre livro (obra) e exemplar (cópia física); (ii) desenvolver uma API REST com autenticação baseada em tokens JWT; (iii) implementar controle de acesso por perfis (Administrador, Funcionário e Aluno); (iv) construir uma interface web responsiva para os diferentes perfis de usuário; e (v) publicar a aplicação em ambiente de produção acessível publicamente.

A justificativa para o projeto é dupla. Do ponto de vista social, o sistema pode apoiar a rotina da biblioteca escolar, liberando tempo dos funcionários e oferecendo aos alunos uma interface mais direta para consulta ao acervo. Do ponto de vista acadêmico, o projeto serve como exercício prático de integração entre tecnologias amplamente utilizadas na indústria de software, consolidando competências em desenvolvimento back-end, front-end, banco de dados, segurança de aplicações e publicação em nuvem.

O relatório organiza-se da seguinte forma. A seção 2.1 apresenta a especificação funcional do sistema. A seção 2.2 descreve a arquitetura adotada. A seção 2.3 detalha as tecnologias empregadas. A seção 2.4 apresenta o modelo de dados. A seção 2.5 documenta os principais endpoints da API. A seção 2.6 discute a metodologia de desenvolvimento adotada. A seção 2.7 trata da implantação do sistema. Por fim, a seção 3 apresenta as considerações finais, incluindo limitações identificadas e possibilidades de trabalhos futuros.

2 DESENVOLVIMENTO

Esta seção apresenta os aspectos técnicos do sistema Biblioteca Monsa, desde a especificação funcional até os detalhes de implantação. A estrutura segue a abordagem de relatórios técnicos, partindo de uma visão de alto nível do escopo e refinando gradualmente para aspectos de implementação.

2.1 Especificação do Projeto

O sistema Biblioteca Monsa é uma aplicação web composta por uma interface de usuário em React e uma API REST em Spring Boot, apoiadas por um banco de dados PostgreSQL. O sistema atende três perfis de usuário, cada um com permissões distintas, conforme descrito a seguir.

2.1.1 Perfis de usuário

O sistema contempla três perfis de acesso, persistidos como registros da tabela `tb_roles` e mapeados para as constantes `ROLE_ADMIN`, `ROLE_FUNCIONARIO` e `ROLE_ALUNO`:

- a) Administrador: responsável pelo cadastro e manutenção do acervo (livros, autores, gêneros, catalogação), pelo cadastro de exemplares físicos e pela administração de usuários do sistema;
- b) Funcionário: responsável pelas operações do dia a dia da biblioteca, como registro de empréstimos, devoluções, consulta a usuários e gerenciamento de solicitações de renovação enviadas pelos alunos;
- c) Aluno: usuário final da biblioteca, capaz de consultar o acervo, visualizar seu empréstimo ativo e seu histórico recente, realizar reservas e enviar solicitações de renovação.

2.1.2 Funcionalidades principais

O escopo funcional do sistema foi segmentado nos seguintes módulos:

Cadastro de acervo: registro de livros com título, editora, CDD, localização física, descrição, imagem de capa e vínculos com autor, gênero e catalogação. Cada livro possui contadores de total de exemplares, quantidade atualmente disponível e número acumulado de empréstimos.

Controle de exemplares: cada cópia física do livro é representada individualmente na tabela `tb_exemplar`, identificada por um código e associada a um

status (DISPONIVEL, EMPRESTADO ou RESERVADO). Essa separação entre livro (obra) e exemplar (cópia) permite que o sistema rastreie qual cópia específica está com qual aluno.

Empréstimos: registro de saída de exemplares, com data de empréstimo, data prevista de devolução (padrão de sete dias a partir da data de empréstimo) e data efetiva de devolução. O empréstimo possui três estados possíveis: ATIVO, ATRASADO e DEVOLVIDO. A transição para ATRASADO é feita por uma rotina agendada, descrita mais adiante.

Solicitações de renovação: o aluno não renova seu empréstimo diretamente. Em vez disso, envia uma solicitação de renovação, que fica pendente até que um funcionário aprove ou rejeite. As solicitações são persistidas em tabela própria, com status PENDENTE, APROVADA ou REJEITADA, data de solicitação e data de resposta.

Reservas: um aluno pode reservar um livro indisponível. A reserva possui cinco estados possíveis: ATIVA (na fila, aguardando devolução de algum exemplar), DISPONIVEL (livro devolvido, aguardando retirada), CONCLUIDA (aluno retirou o livro), EXPIRADA (não retirou no prazo) e CANCELADA (cancelamento manual).

Pesquisa e consulta: o acervo pode ser consultado publicamente, sem autenticação, tanto para listagem completa quanto para busca por título, autor, gênero ou catalogação. A listagem de livros populares é ordenada pelo contador de empréstimos acumulados.

2.1.3 Fluxo de empréstimo

O registro de um novo empréstimo segue os seguintes passos, conforme implementado no método `insertEmprestimo` da classe `EmprestimoService`:

1. O funcionário seleciona o livro e, opcionalmente, um exemplar específico. Se nenhum exemplar for indicado, o sistema seleciona automaticamente o primeiro exemplar disponível, ordenado pelo código;
2. O sistema valida se a quantidade disponível do livro é maior que zero; caso contrário, a operação é rejeitada com mensagem de erro;
3. O exemplar selecionado tem seu status alterado para EMPRESTADO;
4. É criado um novo registro de empréstimo com status ATIVO, data de empréstimo igual à data informada (ou à data atual, se omitida) e data de devolução sete dias após a data de empréstimo;

5. O contador de quantidade disponível do livro é decrementado e o contador acumulado de empréstimos é incrementado em uma unidade.

No ato da devolução, o exemplar retorna ao status DISPONIVEL, a quantidade disponível do livro é incrementada e o empréstimo passa ao status DEVOLVIDO, com a data de devolução efetiva registrada.

2.1.4 Rotinas agendadas

O sistema utiliza o mecanismo de agendamento do Spring (anotação @Scheduled) para executar duas rotinas periódicas de manutenção de estado:

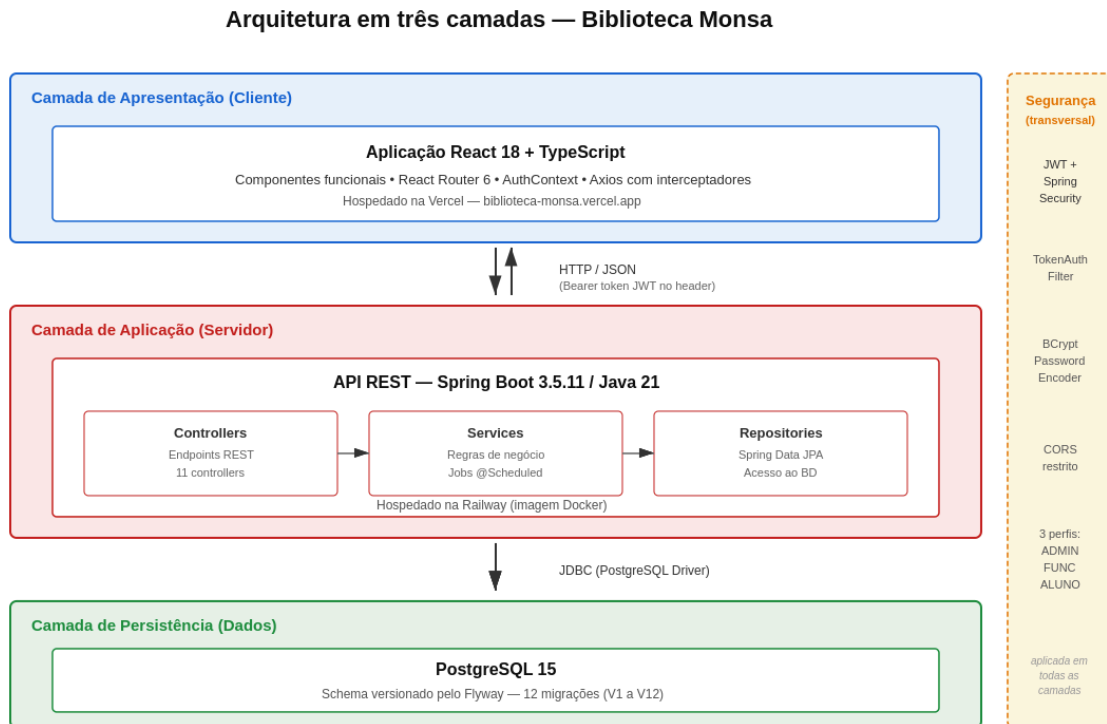
a) Marcação de empréstimos atrasados: executada diariamente à meia-noite, varre todos os empréstimos com status ATIVO e altera para ATRASADO aqueles cuja data prevista de devolução já tenha sido ultrapassada;

b) Expiração de reservas: executada a cada hora, verifica as reservas que estão no estado DISPONIVEL há mais tempo do que o permitido e as marca como EXPIRADA, liberando o exemplar para o próximo aluno da fila, caso exista.

2.2 Arquitetura do Sistema

A aplicação segue uma arquitetura em três camadas distribuídas: camada de apresentação no lado do cliente (front-end React), camada de aplicação no lado do servidor (API REST em Spring Boot) e camada de persistência (banco de dados PostgreSQL). A comunicação entre front-end e back-end ocorre por meio do protocolo HTTP, com payloads em formato JSON. A comunicação entre back-end e banco de dados utiliza JDBC, abstraído pelo Hibernate e pelo Spring Data JPA. A Figura 1 ilustra a organização das três camadas e a posição transversal dos mecanismos de segurança.

Figura 1 – Arquitetura em três camadas da Biblioteca Monsa



Fonte: elaborado pelo autor.

2.2.1 Organização do back-end

O back-end foi organizado em pacotes que refletem as camadas lógicas da aplicação. O pacote controller concentra os pontos de entrada da API REST. O pacote service contém as regras de negócio e coordena as operações entre múltiplos repositórios. O pacote repository contém as interfaces que estendem JpaRepository, responsáveis pelo acesso ao banco de dados. O pacote entity contém as classes que representam as tabelas do banco. O pacote dto agrupa os objetos de transferência de dados, separados em subpastas por entidade (AutorDtos, LivroDtos, EmprestimoDtos, entre outros). O pacote security contém a configuração do Spring Security, o filtro de autenticação por token e o serviço de geração e validação dos tokens JWT.

2.2.2 Autenticação e controle de acesso

A autenticação é implementada por meio de tokens JWT. O fluxo ocorre da seguinte forma: o usuário envia suas credenciais (e-mail e senha) para o endpoint POST /auth; o back-end valida as credenciais utilizando o BCryptPasswordEncoder para comparar com o hash armazenado no banco e, em caso de sucesso, retorna um token JWT assinado no formato "Bearer <token>". Nas requisições

subsequentes, o front-end inclui o token no cabeçalho Authorization. A classe `TokenAuthenticationFilter` intercepta cada requisição, valida o token por meio do `TokenService` e, sendo válido, popula o contexto de segurança do Spring com os dados do usuário.

O controle de autorização é configurado de forma declarativa na classe `SecurityConfiguration`, por meio de regras baseadas em método HTTP e padrão de URL. A matriz de permissões implementada é resumida no Quadro 1.

Quadro 1 – Matriz de permissões por recurso

Recurso	Leitura (GET)	Escrita (POST/PUT/DELETE)
/livro	Público	Administrador
/autor, /genero, /catalogacao	Público	Administrador
/exemplar	Funcionário e Administrador	Administrador
/emprestimo/minha-conta	Aluno, Funcionário e Administrador	—
/emprestimo (demais rotas)	Funcionário e Administrador	Funcionário e Administrador
/reserva	Autenticado (Aluno+)	Autenticado (Aluno+)
/solicitacoes-renovacao (POST)	—	Autenticado (Aluno+)
/solicitacoes-renovacao (GET/PUT)	Funcionário e Administrador	Funcionário e Administrador
/user	Funcionário e Administrador	Funcionário e Administrador

Fonte: elaborado pelo autor.

2.2.3 Organização do front-end

O front-end é uma aplicação React composta por componentes funcionais, que utilizam hooks (`useState`, `useEffect` e `useContext`) para gerenciamento de estado local. O estado global de autenticação é gerenciado por um contexto dedicado (`AuthContext`), que expõe o usuário autenticado e as operações de login e logout para toda a árvore de componentes. O token JWT e os dados do usuário são persistidos no `localStorage` do navegador, sob as chaves `auth_token` e `user`, de modo que a sessão seja mantida entre recargas da página.

A comunicação com o back-end é centralizada no arquivo `services/api.ts`, onde é configurada uma instância do `Axios` com interceptadores. O interceptador de

requisição adiciona automaticamente o cabeçalho Authorization a partir do token armazenado. O interceptador de resposta trata respostas com status 401, removendo o token inválido e redirecionando o usuário para a tela de login. O roteamento entre as páginas é feito com React Router, versão 6.

2.3 Tecnologias Utilizadas

As tecnologias adotadas no projeto foram escolhidas por sua maturidade, pela quantidade de documentação disponível e pela sua relevância no mercado de desenvolvimento de software. Esta seção descreve as principais ferramentas utilizadas em cada camada da aplicação.

2.3.1 Back-end

O back-end foi desenvolvido em Java 21, com o framework Spring Boot, versão 3.5.11, que simplifica a configuração de aplicações Spring ao fornecer autoconfiguração, servidor embutido e gerenciamento de dependências opinativo. As principais dependências utilizadas são:

Spring Web: para construção da API REST e do servidor HTTP embutido (Tomcat);

Spring Data JPA: para abstração da camada de persistência, com geração automática de consultas a partir da assinatura de métodos nos repositórios;

Spring Security: para configuração declarativa das regras de autenticação e autorização;

Spring Validation: para validação de dados de entrada por meio de anotações Bean Validation nos DTOs de requisição;

JWT 0.11.5: para geração e validação dos tokens JWT;

Flyway: para controle de versão do schema do banco de dados, por meio de scripts SQL versionados em src/main/resources/db/migration;

Lombok: para redução do código repetitivo (boilerplate) em entidades e DTOs, por meio de anotações como `@Getter`, `@Setter`, `@AllArgsConstructor` e `@NoArgsConstructor`;

SpringDoc OpenAPI: para geração automática da documentação da API no formato OpenAPI 3, com interface interativa acessível em /swagger-ui.html.

2.3.2 Front-end

O front-end utiliza React, versão 18, com TypeScript 4.9, proporcionando tipagem estática e detecção de erros em tempo de compilação. As principais bibliotecas utilizadas são:

React Router DOM 6: para roteamento declarativo entre as páginas da aplicação;

Axios 1.6: como cliente HTTP, com suporte a interceptadores de requisição e resposta;

Chart.js e react-chartjs-2: para a renderização de gráficos na página de relatórios;

Sass: como pré-processador CSS, utilizado em conjunto com arquivos de variáveis para padronização visual.

O projeto segue a estrutura padrão do Create React App, organizado em diretórios dedicados: components (componentes reutilizáveis, subdivididos por domínio), pages (páginas completas), context (contextos globais, como AuthContext, LoadingContext e ToastContext), hooks (hooks customizados), services (comunicação com a API), types (definições de tipos TypeScript) e utils (utilitários diversos, como a função debounce).

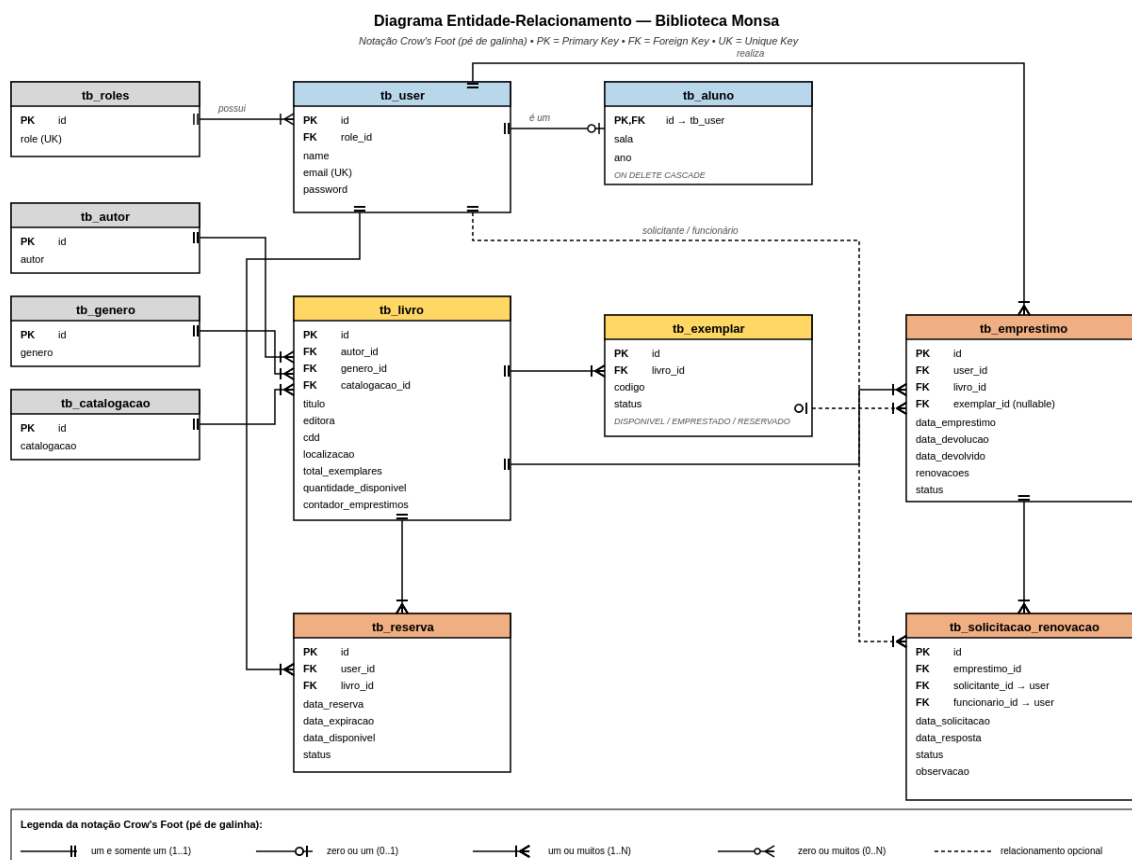
2.3.3 Banco de dados e infraestrutura

O banco de dados adotado é o PostgreSQL, um sistema gerenciador de banco de dados relacional de código aberto, amplamente utilizado em ambientes de produção. O gerenciamento de versões do schema é feito por meio do Flyway, com doze migrações versionadas (V1 a V12) no momento da redação deste relatório. O controle de versão do código-fonte é realizado com Git, hospedado no GitHub. O build do back-end é feito com Maven, e o do front-end com npm e react-scripts.

2.4 Modelo de Dados

O modelo de dados relacional do sistema é composto por onze tabelas principais, criadas e mantidas por meio de doze scripts de migração do Flyway. Esta seção descreve as tabelas, seus principais atributos e os relacionamentos entre elas. A Figura 2 apresenta o diagrama entidade-relacionamento do sistema.

Figura 2 – Diagrama Entidade-Relacionamento da Biblioteca Monsa



Fonte: elaborado pelo autor.

2.4.1 Tabelas do domínio

O Quadro 2 apresenta de forma resumida as tabelas do banco de dados e sua finalidade.

Quadro 2 – Tabelas do banco de dados

Tabela	Finalidade
tb_autor	Cadastro de autores. Relacionamento N:1 com tb_livro.
tb_catalogacao	Classificações bibliográficas (CDD). Relacionamento N:1 com tb_livro.
tb_genero	Gêneros literários. Relacionamento N:1 com tb_livro.
tb_livro	Cadastro de obras do acervo, com título, editora, CDD, localização, descrição, URL da capa, total de exemplares, quantidade disponível e contador acumulado de empréstimos.
tb_exemplar	Cópias físicas individuais de cada livro, identificadas por um código e com status próprio (DISPONIVEL, EMPRESTADO, RESERVADO).

Tabela	Finalidade
tb_roles	Catálogo de perfis de acesso: ROLE_ADMIN, ROLE_FUNCIONARIO, ROLE_ALUNO.
tb_user	Cadastro de usuários, com nome, e-mail, senha e vínculo com um perfil da tabela tb_roles.
tb_aluno	Extensão da tabela tb_user para dados específicos de alunos (sala e ano). Relacionamento de herança por chave compartilhada, com exclusão em cascata.
tb_emprestimo	Registro de empréstimos, com datas, número de renovações, status (ATIVO, ATRASADO, DEVOLVIDO) e vínculos com usuário, livro e exemplar.
tb_reserva	Registro de reservas, com datas e status (ATIVA, DISPONIVEL, CONCLUIDA, EXPIRADA, CANCELADA).
tb_solicitacao_renovacao	Solicitações de renovação enviadas pelos alunos, com status (PENDENTE, APROVADA, REJEITADA), datas de solicitação e resposta, e referência ao funcionário responsável pela decisão.

Fonte: elaborado pelo autor.

2.4.2 Relacionamentos e integridade

Os principais relacionamentos entre as tabelas são:

- a) tb_livro associa-se a tb_autor, tb_genero e tb_catalogacao por meio de chaves estrangeiras (cardinalidade N:1);
- b) tb_exemplar associa-se a tb_livro (N:1), representando a relação “um livro possui muitos exemplares”;
- c) tb_emprestimo associa-se a tb_user, tb_livro e tb_exemplar (N:1 em todas). O campo exemplar_id é opcional (nullable), para preservar compatibilidade com empréstimos anteriores à introdução da tabela tb_exemplar;
- d) tb_reserva associa-se a tb_user e tb_livro (N:1);
- e) tb_solicitacao_renovacao associa-se a tb_emprestimo, a tb_user (como solicitante) e, opcionalmente, a tb_user novamente (como funcionário responsável pela decisão);
- f) tb_aluno é uma extensão de tb_user por meio de chave compartilhada, com exclusão em cascata: ao excluir um usuário que seja aluno, o registro correspondente em tb_aluno é automaticamente removido.

O validador do Hibernate é utilizado em modo “validate” (propriedade `spring.jpa.hibernate.ddl-auto = validate`), o que assegura, no momento da

inicialização da aplicação, que as classes de entidade estejam em conformidade com o schema gerenciado pelo Flyway.

2.4.3 Migração retroativa na introdução de exemplares

A introdução da tabela `tb_exemplar`, no script V12, foi acompanhada de uma migração de dados retroativa. O script gera, para cada livro já cadastrado, um número de exemplares igual ao valor de `total_exemplares`, nomeados sequencialmente (001, 002, 003 e assim por diante). A diferença entre `total_exemplares` e `quantidade_disponivel` determina quantos exemplares são criados com status EMPRESTADO. Em seguida, uma segunda rotina vincula cada empréstimo então em andamento a um dos exemplares emprestados, preservando a rastreabilidade do histórico.

2.5 API e Endpoints

A API é organizada em onze controllers, cada um responsável por um recurso do domínio. Esta seção apresenta, de forma resumida, os principais endpoints agrupados por recurso. A documentação completa, com os esquemas de requisição e resposta, é gerada automaticamente pelo SpringDoc OpenAPI e está acessível em `/swagger-ui.html` durante a execução da aplicação.

2.5.1 Autenticação

`POST /auth` — autentica um usuário. Recebe um corpo JSON com os campos email e password e retorna um token no formato “Bearer <token>”;

`POST /auth/registrar` — registra um novo usuário. Utilizado para cadastro inicial de contas.

2.5.2 Acervo (livros e entidades relacionadas)

`GET /livro/todos` — retorna a lista completa de livros;

`GET /livro/id/{id}` — retorna um livro específico pelo identificador;

`GET /livro/buscar/{filtro}/{termo}` — busca livros por um filtro (titulo, autor, genero ou catalogacao) e um termo;

`GET /livro/populares` — retorna os livros ordenados pelo contador acumulado de empréstimos;

`POST /livro`, `PUT /livro/{id}`, `DELETE /livro/{id}` — operações de escrita restritas ao perfil Administrador;

`GET/POST/PUT/DELETE` em `/autor`, `/genero`, `/catalogacao` — operações análogas para as entidades de domínio vinculadas ao livro. As operações de leitura são públicas (para permitir o funcionamento do autocompletar sem autenticação), e as de escrita são restritas ao perfil Administrador.

2.5.3 Exemplos

`GET /exemplar/livro/{livroId}` — lista todos os exemplares de um livro;

`GET /exemplar/livro/{livroId}/disponiveis` — lista apenas os exemplares com status DISPONIVEL;

`GET /exemplar/livro/{livroId}/sugestao` — retorna o próximo exemplar sugerido para empréstimo (primeiro DISPONIVEL ordenado por código);

`POST /exemplar/livro/{livroId}/adicionar` — adiciona um novo exemplar ao livro informado (restrito ao perfil Administrador).

2.5.4 Empréstimos

`POST /emprestimo` — registra um novo empréstimo;

`PUT /emprestimo/renovar/{id}` — renova um empréstimo, estendendo a data de devolução em sete dias e incrementando o contador de renovações;

`PUT /emprestimo/devolver/{id}` — registra a devolução de um empréstimo;

`GET /emprestimo/minha-conta/{email}` — endpoint utilizado pelo aluno autenticado para consultar seu empréstimo em curso (ATIVO ou ATRASADO) e os últimos quatro empréstimos do histórico;

`GET /emprestimo/todos`, `/emprestimo/id/{id}`, `/emprestimo/user/{nome}`, `/emprestimo/livro/{titulo}`, `/emprestimo/status/{status}`, `/emprestimo/devolucao/{data}` — consultas de gestão utilizadas pelos perfis Funcionário e Administrador.

2.5.5 Solicitações de renovação

`POST /solicitacoes-renovacao/{email}` — aluno solicita a renovação do próprio empréstimo;

GET /solicitacoes-renovacao/pendentes — funcionário consulta a fila de solicitações ainda não respondidas;

PUT /solicitacoes-renovacao/{id}/aprovar/{email} — funcionário aprova uma solicitação, o que dispara a renovação efetiva do empréstimo associado;

PUT /solicitacoes-renovacao/{id}/rejeitar/{email} — funcionário rejeita uma solicitação, registrando observação e data de resposta.

2.5.6 Reservas

POST /reserva — cria uma nova reserva para um livro;

DELETE /reserva/{reservaId} — cancela uma reserva existente;

GET /reserva/todos, /reserva/username/{name}, /reserva/useremail/{email}, /reserva/livro/{livroId} — consultas de reservas por diferentes critérios.

2.6 Metodologia de Desenvolvimento

O desenvolvimento do sistema foi conduzido de forma iterativa, com inspiração nos princípios das metodologias ágeis, em especial do Scrum e do Kanban. O autor atuou como desenvolvedor único, sem equipe formal, mas organizou o trabalho em ciclos curtos de entrega (sprints), cada um dedicado a um conjunto de funcionalidades bem definido. O controle de tarefas foi apoiado por um quadro kanban, com colunas de backlog, em andamento, em revisão e concluído.

O fluxo típico de cada sprint consistia nas seguintes etapas: (i) seleção das funcionalidades a serem entregues; (ii) modelagem do impacto no banco de dados, resultando em novos scripts de migração quando necessário; (iii) implementação das entidades, DTOs, repositórios, services e controllers; (iv) implementação das telas e integrações no front-end; e (v) revisão manual das funcionalidades implementadas por meio da interface Swagger UI e da própria aplicação.

O controle de versão foi feito com Git, com commits pequenos e descritivos, organizados em ramos (branches) dedicados a cada funcionalidade. Durante parte do desenvolvimento, em especial nas etapas de refatoração e documentação do código, o autor também utilizou assistência de ferramentas de inteligência artificial (Claude Code), mantendo-se como revisor final de todas as alterações produzidas.

Esta é uma descrição honesta do processo adotado. O projeto não conta, no momento da redação deste relatório, com suíte de testes automatizados, nem com pipeline de integração contínua. A ausência desses elementos é reconhecida como uma limitação, discutida na seção de considerações finais.

2.7 Implantação

O sistema possui dois ambientes: desenvolvimento local e produção.

2.7.1 Ambiente de desenvolvimento

Em ambiente local, o back-end é executado diretamente a partir da IDE ou por meio do Maven, utilizando o servidor Tomcat embutido na porta 8080. O banco de dados PostgreSQL é executado localmente (com configuração padrão apontando para localhost:5432, usuário postgres e banco gerenciador_biblioteca). O front-end é executado por meio do comando `npm start`, que inicia o servidor de desenvolvimento do Create React App na porta 3000, com um proxy configurado para encaminhar as requisições da API ao back-end local. Os scripts de migração do Flyway são executados automaticamente na inicialização do back-end, criando ou atualizando o schema quando necessário.

2.7.2 Ambiente de produção

O sistema está publicado em produção em dois provedores de nuvem distintos:

- a) **Front-end:** hospedado na Vercel, no endereço `https://biblioteca-monsa.vercel.app`. A Vercel realiza o build do projeto React a cada push no ramo principal do repositório, disponibilizando o resultado por meio de uma rede de distribuição de conteúdo (CDN);
- b) **Back-end:** hospedado na Railway, executado a partir de uma imagem Docker construída a partir do Dockerfile presente na raiz do repositório. A Railway provê também uma instância de PostgreSQL, cujos dados de conexão são injetados como variáveis de ambiente na aplicação.

As variáveis de ambiente configuradas em produção incluem o segredo de assinatura dos tokens JWT (`jwt.secret`), o tempo de expiração dos tokens (`JWT_EXPIRATION`) e a porta do servidor (`PORT`, provida automaticamente pela Railway). O CORS é configurado para aceitar requisições apenas a partir do

endereço do front-end em produção e do ambiente de desenvolvimento local (localhost:3000).

3 CONSIDERAÇÕES FINAIS

O desenvolvimento do sistema Biblioteca Monsa permitiu colocar em prática, de forma integrada, conhecimentos adquiridos ao longo do curso técnico em Desenvolvimento de Sistemas. A construção envolveu todas as camadas de uma aplicação web moderna: modelagem de um banco de dados relacional; implementação de uma API REST com autenticação e controle de acesso por perfis; desenvolvimento de uma interface web em React com tipagem estática; e publicação em ambiente de produção em dois provedores de nuvem distintos.

Os objetivos específicos definidos na introdução foram, em sua maioria, alcançados. O domínio foi modelado em um banco relacional que distingue livro (obra) e exemplar (cópia), o que permite o rastreamento individual das cópias emprestadas. A API REST foi implementada, com autenticação baseada em JWT e controle de acesso granular por perfis e métodos HTTP. A interface web foi construída com páginas diferenciadas para os três perfis de usuário. A aplicação foi publicada e está acessível publicamente.

Há, entretanto, limitações importantes que devem ser reconhecidas. A principal delas é a ausência de uma suíte de testes automatizados. O projeto possui, no momento, apenas o teste padrão gerado pelo Spring Boot (verificação de carga do contexto), e nenhuma bateria de testes unitários ou de integração foi desenvolvida. Em consequência, o processo de evolução do código depende de revisão manual das funcionalidades e é mais suscetível a regressões. Uma segunda limitação é a ausência de pipeline de integração contínua, o que significa que não há verificação automática de build ou execução de testes a cada alteração. Uma terceira limitação é a cobertura incompleta da documentação de requisitos não funcionais, como requisitos de desempenho sob carga e de disponibilidade.

Como possibilidades de trabalhos futuros, identificam-se as seguintes frentes: (i) implementação de uma suíte de testes automatizados, iniciando pelas regras de negócio dos services de empréstimos, reservas e solicitações de renovação; (ii) configuração de um pipeline de integração contínua no GitHub Actions, para execução de testes e verificação de build a cada pull request; (iii) implementação de um módulo de notificações ao aluno (por e-mail ou no próprio sistema) para alertas de proximidade de vencimento do empréstimo; (iv) adição de métricas e logs estruturados, para acompanhamento do comportamento da aplicação em produção;

e (v) implementação de uma estratégia de backup automatizado do banco de dados em produção.

Do ponto de vista da formação do autor, o projeto consolidou competências em desenvolvimento full-stack e na integração entre tecnologias amplamente utilizadas no mercado. O processo também ofereceu experiência prática em decisões de modelagem que não são evidentes em exercícios acadêmicos típicos, como a refatoração para a introdução de exemplares físicos individuais (migração V12) e o desenho do fluxo de solicitação e aprovação de renovações. O código-fonte está disponível publicamente no endereço https://github.com/sandersonElias/Gerenciador_Biblioteca, de modo que outras instituições ou estudantes possam consultá-lo, adaptá-lo ou utilizá-lo como referência.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 6023: informação e documentação: referências: elaboração. 3. ed. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 6024: informação e documentação: numeração progressiva das seções de um documento: apresentação. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 6027: informação e documentação: sumário: apresentação. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 6028: informação e documentação: resumo: apresentação. Rio de Janeiro: ABNT, 2021.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 10520: informação e documentação: citações em documentos: apresentação. 2. ed. Rio de Janeiro: ABNT, 2023.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 10719: informação e documentação: relatório técnico e/ou científico: apresentação. 4. ed. Rio de Janeiro: ABNT, 2015.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 14724: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

FIELDING, Roy Thomas. Architectural styles and the design of network-based software architectures. 2000. Tese (Doutorado em Information and Computer Science) – University of California, Irvine, 2000.

FOWLER, Martin. Patterns of enterprise application architecture. Boston: Addison-Wesley, 2002.

GAMMA, Erich et al. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000.

JONES, Michael; BRADLEY, John; SAKIMURA, Nat. RFC 7519: JSON Web Token (JWT). Internet Engineering Task Force, 2015. Disponível em: <https://datatracker.ietf.org/doc/html/rfc7519>. Acesso em: 24 abr. 2026.

ORACLE. Java Platform, Standard Edition Documentation. 2024. Disponível em: <https://docs.oracle.com/en/java/>. Acesso em: 24 abr. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL 15 Documentation. 2024. Disponível em: <https://www.postgresql.org/docs/15/>. Acesso em: 24 abr. 2026.

REACT. React Documentation. 2024. Disponível em: <https://react.dev/>. Acesso em: 24 abr. 2026.

RED HAT. Hibernate ORM User Guide. 2024. Disponível em: <https://hibernate.org/orm/documentation/>. Acesso em: 24 abr. 2026.

SPRING. Spring Boot Reference Documentation. 2024. Disponível em: <https://docs.spring.io/spring-boot/docs/current/reference/html/>. Acesso em: 24 abr. 2026.

SPRING. Spring Security Reference. 2024. Disponível em: <https://docs.spring.io/spring-security/reference/>. Acesso em: 24 abr. 2026.